

Method Syntax and List Iteration

CSCI 1101

2024-02-22

Method Syntax

So far we have been using python's **standard function syntax**:

```
<func_name>(<first_arg> , ... , <last_arg>)
```

For *some* functions that live in a module, like `str` or `list`, if the first argument has the type of that module then there is an alternative syntax for calling them.

In **method syntax** we move the first argument to replace the module, so:

```
<module>.<func_name>(<first_arg> , <rest_args>)
```

becomes:

```
<first_arg>.<func_name>(<rest_args>)
```

Python can calculate the module automatically from the type of the first argument.

Object Oriented Programming

Method syntax is most useful when we think of the first argument to a function as its **principal argument**, and think of calling a function as passing a **message** to that object asking it to do something.

For example:

```
list.reverse(my_list)
```

becomes:

```
my_list.reverse()
```

We can think of this as sending a message to `my_list` asking it to reverse itself.

This style of programming is called **object oriented programming**.

We will study it in more detail later in the course.

Activity: Method Syntax

Translate the following function calls from standard syntax to method syntax:

- `list.sort(my_list)`
- `list.append(my_list , item)`
- `list.pop(my_list , index)`
- `str.upper("hello")`

Translate the following function calls from method syntax to standard syntax:

- `my_list.insert(index , item)`
- `my_list.extend(other_list)`
- `my_list.copy()`
- `"42".isdigit()`

List Iteration

A very common programming task is to do something with each element of a list.

One way to do this in python is with the **list iteration statement** or “for loop”.

The syntax is:

```
for <loop_variable> in <list> :  
    <loop_body>
```

where `list` is any list, `loop_variable` is a new variable name that you choose, and `loop_body` is any nonempty sequence of statements (i.e. a **block**).

When python encounters a for loop it runs the loop body once for each element in the list, automatically assigning the loop variable to the corresponding list element each time.

Iteration Example: Printing

Suppose that we want to write a function that takes a list of things and prints a string representation of each one in turn.

We can do this using a for loop:

```
def print_each(items) :  
    """  
    signature: (list a) --> NoneType  
    prints each item in a list  
    """  
    for item in items :  
        print(str(item))
```

The Accumulator Pattern

A very general pattern for computing with data in a list is the **accumulator pattern**.

We use an **accumulator** variable to keep track of the information that we want to learn about the list and use a loop to update this information for each list element.

The general structure of the accumulator pattern is:

```
acc = <initial_value>
for item in <list> :
    acc = <update_function>
<do_something_with_acc>
```

The **initial value** of the accumulator should be whatever is the “right answer” if the list is empty.

The **update function** is an expression involving the current accumulator value and current list element that specifies the next accumulator value.

Accumulator Example: Summing

Say we want to sum the numbers in a list:

- What should be the sum when the list is empty?
This is the initial value of the accumulator.
- If we know the list sum so far and the current number, what is the new sum?
This is the update function.

```
def sum_list(nums) :  
    sum = 0                # initialize accumulator  
    for num in nums :     # iterate over list  
        sum = sum + num    # update accumulator  
    return sum            # use the result
```


Variable Update Assignment Notation

A common task is assigning a variable a new value based on its current value.

For many operators `_*_` we can write:

```
var *= val
```

as shorthand for:

```
var = var * val
```

For example, to double the value of an integer variable `x`:

```
x *= 2
```

To pluralize the value of a string variable `thing`:

```
thing += "s"
```

Activity: Counting Even Numbers

Suppose we want to know how many of the numbers in a list are even.

```
def count_evens(nums) :  
    """signature: (list int) --> int"""
```

Let's try doing this using the accumulator pattern.

- How many even numbers are there in the empty list?
This is the initial value of the accumulator.
- If we know the even count of the list so far and the current number, then what is the new even count?
This is the update function for the accumulator.

Activity: List Reversing

Suppose we want to write a list-reversing function for ourselves.

```
def reverse_list(items) :  
    """signature: (list a) --> list a"""
```

Let's try doing this using the accumulator pattern.

- What is the reversed version of the empty list?
This is the initial value of the accumulator.
- If we have the reversed version of the list so far and the current element, then how do we extend the reversed list?
This is the update function for the accumulator.

To Do

Finish *Project 1 - The Turtle Module* on CodeRunner by midnight tomorrow.

Finish *Homework 5: Lists and Loops* on CodeRunner by 11:00AM next Thursday.